Day 124 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 18, Day 6 reference

Watch: Singular Value Decomposition (SVD): Overview (Steve Brunton) | Singular Value Decomposition (the SVD) (MIT OpenCourseWare)

`day124_svd.py`, segment by segment, in the order `main()` uses things. Two SVD routes, the rank-k approximation, matrix completion and the pseudoinverse are new; Day 123's pca_fit is imported for the comparison. np.linalg.svd and np.linalg.lstsq are the checks. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: A = any matrix · U, V = left and right singular vectors (columns) · s = singular values, largest first · mask = True where an entry is known · k = how many triplets are kept

Segment 0 — Setup

Imports and settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day123_pca as d123
```

- `from sklearn.datasets import load_sample_image` — photographs that ship with scikit-learn; reading the JPEG needs the Pillow library.
- `import day123_pca as d123` — Day 123's eigenvalue route, to compare with the SVD route.

Segment 1 — The SVD

svd_via_eig: the SVD from A^T A

MATH

$$A^T A = V S^2 V^T; \quad u_i = A v_i / s_i$$

CODE

```
def svd_via_eig(A):
    vals, V = np.linalg.eigh(A.T @ A)
    o = np.argsort(vals)[::-1]; vals, V = vals[o], V[:, o]
    s = np.sqrt(np.maximum(vals, 0))
    U = (A @ V) / np.where(s > 0, s, 1)
    return U, s, V
```

- `np.linalg.eigh(A.T @ A)` — $A^T A$ is symmetric, so eigh applies; its eigenvalues are the squared singular values.
- `np.where(s > 0, s, 1)` — avoids dividing by zero for a zero singular value (its u is then meaningless).
- `(A @ V) / ...` — divides each column of A V by its own singular value: broadcasting along the rows.

svd_power: alternating multiplication

MATH

$$u \leftarrow Av/|Av|; \quad v \leftarrow A^T u/|A^T u|; \quad s = |A^T u| \text{ at convergence}$$

CODE

```
def svd_power(A, k, iters=3000, tol=1e-12, seed=0):
    rng = np.random.RandomState(seed); U, s, V = [], [], []
    for _ in range(k):
        v = rng.normal(size=A.shape[1]); v /= np.linalg.norm(v)
        for _ in range(iters):
            u = A @ v - sum(sj * uj * (vj @ v) for sj, uj, vj in zip(s, U, V))
            u /= np.linalg.norm(u)
            w = A.T @ u - sum(sj * vj * (uj @ u) for sj, uj, vj in zip(s, U, V))
            sigma = np.linalg.norm(w); w /= sigma
            done = np.linalg.norm(w - v) < tol
            v = w
            if done:
                break
        U.append(u); s.append(sigma); V.append(v)
    return np.array(U).T, np.array(s), np.array(V).T
```

- `sum(sj * uj * (vj @ v) for ...)` — a generator inside sum(): subtracts the triplets already found, without changing A.
- `sigma = np.linalg.norm(w)` — the length of $A^T u$ before rescaling: the singular value.
- `zip(s, U, V)` — walks three lists in step: one found triplet at a time.

low_rank, complete and pinv_solve

MATH

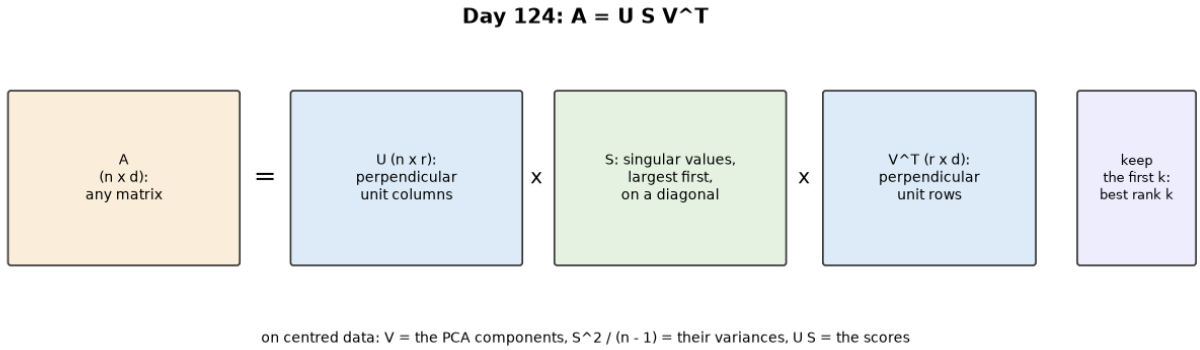
$$A_k = U_k S_k V_k^T; \quad x = V_k S_k^{-1} U_k^T b$$

CODE

```
def low_rank(U, s, V, k):
    return (U[:, :k] * s[:k]) @ V[:, :k].T
def complete(M, mask, k, iters=2000, tol=1e-6):
    X = np.where(mask, M, M[mask].mean())
    for t in range(iters):
        U, s, Vt = np.linalg.svd(X, full_matrices=False)
        L = (U[:, :k] * s[:k]) @ Vt[:k]
        change = np.linalg.norm((L - X)[~mask]) / max(np.linalg.norm(X[~mask]), 1e-12)
        X = np.where(mask, M, L)
        if change < tol:
            break
    return X, t + 1
def pinv_solve(A, b, k=None):
    U, s, Vt = np.linalg.svd(A, full_matrices=False); k = len(s) if k is None else k
    return Vt[:k].T @ ((U[:, :k].T @ b) / s[:k])
```

- `(U[:, :k] * s[:k]) @ V[:, :k].T` — scales each column of U by its singular value, then multiplies: $U_k S_k V_k^T$.
- `np.linalg.svd(X, full_matrices=False)` — the 'thin' SVD: U has only as many columns as there are singular values.
- `np.where(mask, M, L)` — known entries from the data, unknown ones from the fit.
- `(U[:, :k].T @ b) / s[:k]` — the coordinates of b along each u, divided by its stretch.

SYSTEM DIAGRAM



$A = U\,S\,V^T$.

Segment 2 — The experiments

demo_routes: the Hilbert matrix

CODE

```
n = 12; H = 1.0 / (np.arange(n)[:, None] + np.arange(n)[None, :] + 1)
s_ref = np.linalg.svd(H, compute_uv=False); _, s_eig, _ = svd_via_eig(H)
```

- `np.arange(n)[:, None] + np.arange(n)[None, :]` — a column plus a row broadcasts to a table: entry (i, j) = i + j.
- `compute_uv=False` — only the singular values, which is all the comparison needs.

demo_complete: a random half of the pixels

CODE

```
P = photo()[::2, ::2]; rng = np.random.RandomState(0); mask = rng.uniform(size=P.shape) < 0.5
rmse = lambda X: float(np.sqrt(np.mean((X - P)[~mask] ** 2)))
col = np.where(mask, P, (np.where(mask, P, 0).sum(0) / mask.sum(0))[None, :])
```

- `photo()[::2, ::2]` — every second row and column: a quarter of the pixels, for speed.
- `rmse = lambda X: ...[~mask]` — ~ flips True and False: the error is measured only where pixels were hidden.
- `np.where(mask, P, 0).sum(0) / mask.sum(0)` — each column's mean over its known pixels only.

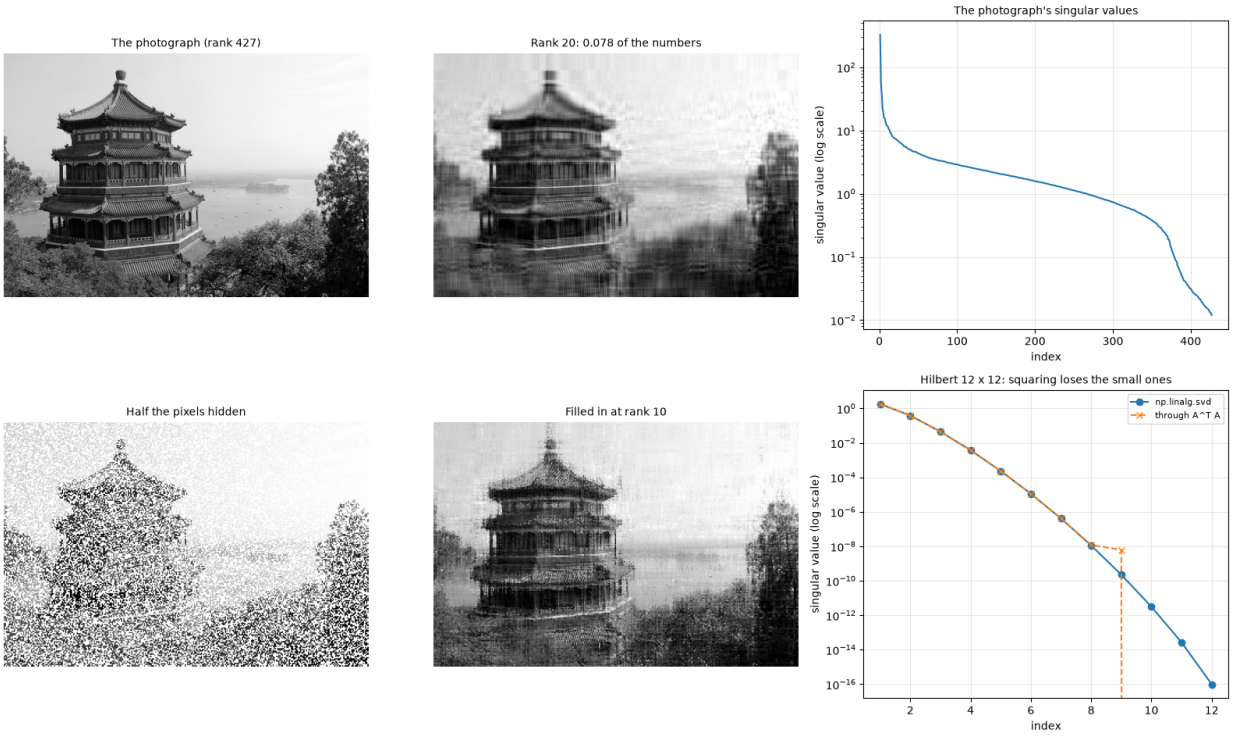
demo_lstsq: which features make the smallest direction

CODE

```
v = np.linalg.svd(A[tr], full_matrices=False)[2][-1]; top = np.argsort(-np.abs(v))[:2]; names =
    ["intercept"] + list(d.feature_names)
print(f"the smallest one's direction is mostly '{names[top[0]]}' ({v[top[0]]:+.2f}) and
    '{names[top[1]]}' ({v[top[1]]:+.2f}); "
      f"their correlation {np.corrcoef(X[tr][:, top[0] - 1], X[tr][:, top[1] - 1])[0, 1]:.3f}: one
          nearly repeats the other")
```

- `np.linalg.svd(...)[2][-1]` — the last row of V^T: the direction of the smallest singular value.
- `np.argsort(-np.abs(v))[:2]` — the two features with the largest weights in that direction.
- `top[0] - 1` — minus 1 because column 0 of A is the intercept, which X does not have.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.svd(A, full_matrices=False)` — the thin SVD: U, s, V^T.
- `compute_uv=False` — singular values only.
- `sum(f(x) for x in xs)` — a generator inside sum().
- `zip(a, b, c)` — walk lists in step.
- `~mask` — flip a True/False array.
- `load_sample_image("china.jpg")` — a bundled photograph, height x width x 3.

End of Day 124 syntax reference. For the experiments and what they showed, see the Day 124 complete guide.